



HW3-1 Report: Deep Reinforcement Learning on Path Tracking

Implementation

TODO 1 & 2 — Policy & Value Networks (`model.py`)

PolicyNet $\pi_\theta(a \mid s)$: Two-layer MLP ($14 \rightarrow 64 \rightarrow 64$) with ReLU activations and orthogonal weight initialization. The output passes through a `DiagGaussian` head that produces a 1D action mean $\mu_\theta(s)$, wrapped in a `FixedNormal` distribution with fixed $\sigma = 0.5$:

$$a \sim \mathcal{N}(\mu_\theta(s), \sigma^2 I)$$

At inference the mode (mean) is used; during training, actions are sampled stochastically.

ValueNet $V_\omega(s)$: Three-layer MLP ($14 \rightarrow 64 \rightarrow 64 \rightarrow 64 \rightarrow 1$) with ReLU activations and orthogonal initialization. Outputs a scalar state-value estimate.

Both networks use orthogonal initialization with `relu` gain to avoid vanishing/exploding gradients at training start.

TODO 3 — Rollout Collection (`env_runner.py`)

`EnvRunner.run` collects $T = 128$ steps across $N = 8$ parallel environments. At each step t :

1. Sample action and log-probability: $a_t \sim \pi_\theta(\cdot \mid s_t)$, record $\log \pi_\theta(a_t \mid s_t)$
2. Estimate value: $v_t = V_\omega(s_t)$
3. Step environment: $s_{t+1}, r_t, d_t = \text{env.step}(a_t)$

After collecting all steps, **Generalized Advantage Estimation (GAE)** computes advantages:

$$\delta_t = r_t + \gamma V(s_{t+1}) - V(s_t)$$

$$\hat{A}_t = \sum_{l=0}^{T-t} (\gamma \lambda)^l \delta_{t+l}$$

with $\gamma = 0.99$, $\lambda = 0.95$. Returns and advantages are flattened to $(T \times N)$ batches for training.

TODO 4 — PPO Update (agent.py)

The clipped surrogate objective:

$$r_t(\theta) = \frac{\pi_\theta(a_t | s_t)}{\pi_{\theta_{\text{old}}}(a_t | s_t)} = \exp(\log \pi_\theta - \log \pi_{\theta_{\text{old}}})$$

$$L^{\text{CLIP}}(\theta) = -E \left[\min \left(r_t(\theta) \hat{A}_t, \text{clip}(r_t(\theta), 1 - \epsilon, 1 + \epsilon) \hat{A}_t \right) \right]$$

The value loss uses clipped value predictions:

$$L^V(\omega) = E \left[\max \left((G_t - V_\omega)^2, (G_t - \text{clip}(V_\omega, V_{\text{old}} \pm \epsilon))^2 \right) \right]$$

Each PPO update runs $K = 4$ epochs over the rollout, shuffling and sampling $M = 64$ -sample mini-batches. Both actor and critic use Adam optimizers with gradient clipping (max norm = 0.5).

TODO 5 — Training Parameters (train.py)

Parameter	Value
N (parallel envs)	8
T (rollout steps)	128
M (batch size)	64
K (epochs)	4
ϵ (clip)	0.2
γ (discount)	0.99
λ (GAE)	0.95
learning rate	10^{-4} (linear decay)
iterations	30,000

CPU vs GPU Training

The baseline `train.py` uses Python multiprocessing to run 8 parallel environments on CPU. We additionally wrote a GPU-accelerated pipeline (`train_gpu.py` , `gpu_env.py` , `gpu_runner.py` , `gpu_agent.py`) that vectorizes the entire environment and rollout collection on GPU tensors.

Architectural Differences

Aspect	CPU (<code>multi_env.py</code>)	GPU (<code>gpu_env.py</code>)
Parallelism	Multiprocessing (IPC pipes)	Vectorized tensor ops
Env count	8	4096
Data location	NumPy arrays on CPU	PyTorch tensors on GPU
Env-to-model transfer	<code>torch.from_numpy()</code> per step	Zero-copy (all GPU)
Path generation	On-demand per reset	Pre-generated pool + background refresh

The PPO algorithm, network architecture, and loss functions are **identical** between CPU and GPU versions. The GPU version is a drop-in replacement that trades development simplicity for higher throughput via pure tensor parallelization.

Performance Comparison

Measured at the same training iteration (iter 780, 20 iterations of training):

Metric	CPU	GPU
FPS	2,967	459,464
Timesteps collected	20,480	10,485,760
Wall time	6.90 s	22.82 s
Timesteps / second	~3K	~460K
Mean return	170.2	228.0

The GPU version achieves **~155x higher throughput** (459K vs 3K FPS). Although each GPU iteration takes slightly longer in wall time (22.8s vs 6.9s) due to processing 4096 envs instead of 8, it collects **512x more timesteps per iteration** (10.5M vs 20K). This means the GPU version sees far more diverse training data per update, leading to faster convergence and higher mean return at the same iteration count.

Results

The agent learns to track random cubic spline paths using angular velocity control. Training converges around iteration 15,000 with mean episode reward reaching ~150-200 and average episode length ~290 steps (close to the 400-step limit), indicating the agent successfully follows most paths to completion.

Thank you TA

This homework continues HW2, but this time the agent the controlling by itself with real time action output. Compared to HW3-2, The most valuable takeaway is how value networks and policy networks are being constructed, especially the initialization methods. The GPU acceleration methods I created makes training finish under 3 minutes, which is more time efficient and I recommend next year's class could try doing it.